KernelScript: Exploring a Typed Unified-Source Programming Model for eBPF

Anonymous Authors

Abstract

eBPF has become a standard mechanism for extending Linux kernel behavior in networking, observability, and security. Yet realistic eBPF applications are not single programs: developers coordinate verifier-constrained kernel code, userspace loaders, shared maps, generated skeletons, and sometimes kernel module code for kfuncs. This paper studies KernelScript, a beta domain-specific language that explores whether these pieces can be represented in one type-checked source model and lowered to conventional libbpf-compatible artifacts. We present a design analysis of KernelScript's unified programming model, then evaluate the public implementation at commit ccb15b4. On our test host, KernelScript's own test suite reports 85 successful suites and 1095 passing tests. Of 44 repository examples, 43 compile from KernelScript source and 41 build into generated C/eBPF projects. A verifier-load matrix then loads 38 of those build-success eBPF objects with bpftool. Successful examples have median 31 lines of KernelScript source and median 472 lines of generated C and build logic, an 11.3x expansion factor. The results show that the compiler centralizes recurring project structure while exposing remaining compatibility limits in struct_ops skeleton generation, ref-counted object ownership, and kernel-BTF-dependent symbols. An experimental compiler-source patch for one map-update lowering choice closes the count microbenchmark gap with hand-written C/eBPF in this harness.

1 Introduction

eBPF gives Linux developers a controlled way to execute application-specific logic in the kernel [6]. It is now used for packet processing, traffic control, tracing, profiling, security policy, and scheduler experimentation. The programming model remains difficult because the artifact boundary is split: an eBPF program is written under verifier constraints, a userspace loader must open and attach it, maps provide shared state, BTF describes kernel types, and newer interfaces such as kfuncs and struct_ops require close coordination between generated skeleton code and kernel headers.

The usual C/libbpf workflow is powerful but verbose [4]. The developer writes kernel-side C, userspace C, Makefiles, pinning logic, map setup, attachment code, and cleanup paths. Libraries in Rust, Go, or Python improve parts of the workflow [1, 3], but they usually preserve the split between the eBPF program and the controller. Tracing languages such as bpftrace provide a higher-level experience, but intentionally narrow the target domain. The systems question is therefore not whether eBPF needs another surface syntax. The question is whether a language can encode cross-boundary eBPF application structure in a compiler-checkable way that remains compatible with the existing kernel toolchain.

1.1 Why a Unified Source Model?

A single source file is not automatically better than multiple files. The research value comes from making cross-file invariants visible to the compiler. For example, a map declaration is simultaneously a kernel data structure, a userspace file descriptor lookup, a pinning decision, and a type contract between programs. A program reference is simultaneously a source-level function name, a generated BPF program section, a skeleton field, a file descriptor, and possibly a BPF link. In C/libbpf, these relationships are implicit naming conventions and generated-header dependencies. In KernelScript, they can be first-class language values.

This observation motivates the paper's main scientific question: can a typed unified-source model make cross-boundary eBPF application structure explicit to the compiler while preserving the ordinary Linux BPF toolchain path? A tracing-only language such as bpftrace [2] can be concise by excluding XDP, TC, struct_ops, and kfunc workflows. A host-language binding can be ergonomic by focusing on userspace loading. KernelScript attempts a harder point in the design space: preserve multiple eBPF program types and still give the compiler enough structure to generate the loader, maps, and build logic.

KernelScript is a recent DSL that makes this question concrete. It lets developers write eBPF entry points, helper functions, userspace control flow, maps, tail-call style delegation, ring buffers, perf events, kfunc declarations, and kernel-module code in one source language. The compiler lowers this source to eBPF C, userspace C, optional module C, a Makefile, and then the normal libbpf and bpftool path can produce objects, skeletons, and binaries. Because KernelScript is explicitly beta software, its most useful research role today is as a prototype for studying the design space rather than as a production-ready replacement for libbpf.

This paper makes three contributions. First, it formulates KernelScript's central systems idea as a typed unified-source model for multi-artifact eBPF applications, where programs, maps, loaders, pinning choices, and optional kernel-extension code are represented before artifact boundaries are split. Second, it analyzes how the public compiler uses that model to

check selected cross-boundary invariants, including program-handle use, perf-event context signatures, and stack-limit violations before load or attach time. Third, it implements a reproducible artifact evaluation of test coverage, example buildability, generated-code expansion, example marker coverage, static rejection cases, an attach/detach smoke test, XDP microbenchmarks, and a compiler-source patch that isolates one map-update lowering mechanism. The artifact is designed so that a reviewer can rerun the evaluation without changing the KernelScript source tree: all scripts, logs, derived tables, and paper macros are stored outside the cloned repository.

2 Background and Motivation

An eBPF application normally contains at least two components. The first is a restricted kernel program, compiled for the BPF target and accepted only if the kernel verifier can prove memory safety, bounded execution, and legal helper usage. The second is a userspace coordinator that loads objects, finds maps, sets up BPF links, handles signals, and tears resources down. The two components communicate through maps, ring buffers, perf events, or global data.

This split creates three kinds of complexity. **Lifecycle complexity** arises because operations must occur in a valid order: an object must be opened, loaded, attached, read, and detached with type-appropriate APIs. **Representation complexity** appears when the same concept, such as a map or a typed context field, must be represented in source C, generated skeleton headers, userspace lookup code, and BTF-derived type declarations. **Compatibility complexity** appears because eBPF features evolve quickly: struct_ops, kfuncs, dynptrs, TCX, and scheduler extensions depend on kernel, bpftool, and libbpf versions.

The verifier is essential, but it is intentionally late in the pipeline. A developer may write code, compile it, generate skeletons, link a loader, and only then receive a verifier rejection or an attach-time failure. A DSL can help if it moves some of these checks earlier without hiding the underlying eBPF execution model. For example, a language can know that a function marked `@xdp` expects an `xdp_md` context and returns an XDP action, model maps as typed global variables, and distinguish userspace functions from eBPF functions and kfunc definitions.

3 KernelScript Overview

KernelScript uses attributes to assign functions to execution domains. A function marked `@xdp`, `@tc`, `@probe`, `@tracepoint`, or `@perf_event` becomes an eBPF entry point. A function marked `@helper` is callable from eBPF code. A normal `main` function is compiled into userspace C. A function marked `@kfunc` can trigger kernel-module generation. The source therefore contains the application structure that would otherwise be scattered across multiple files.

Listing 1: Minimal shape of a unified-source eBPF application.

```
include "xdp.kh"

@xdp fn pass_all(ctx: *xdp_md) -> xdp_action {
  return XDP_PASS
}

fn main() -> i32 {
  var prog = load(pass_all)
  attach(prog, "lo", 0)
  detach(prog)
  return 0
}
```

Maps are declared as typed globals rather than as separate C sections and userspace lookup code. A KernelScript program can declare a hash map from `u32` to a struct, use it in an XDP function, and read or update it in userspace. The compiler lowers the declaration to eBPF map definitions, skeleton references, and helper functions. The same pattern applies to ring buffers and perf events: high-level operations remain explicit in source, but the generated code performs the required libbpf calls.

The language also experiments with lifecycle typing. In the source model, `load` returns a program value and `attach` consumes a compatible loaded program and target. The repository contains tests for detach API generation and type errors around program references. The implementation is not a proof of full typestate soundness, but it shows the research direction: lifecycle ordering can be treated as a language-level property rather than only as a runtime convention.

4 Design Principles

Preserve the kernel toolchain. KernelScript does not replace the kernel verifier, clang, bpftool, or libbpf. Instead it generates C and Makefiles that flow through the same toolchain used by C/libbpf projects. This choice gives the prototype immediate access to BTF, skeleton generation, and existing kernel diagnostics. It also means compatibility failures remain visible and measurable.

Make domains explicit. The attribute system separates userspace, eBPF, helper, and kernel-module code. This matters because the same surface syntax cannot mean the same thing in every domain. Userspace can allocate and print freely, eBPF code must obey stack and helper restrictions, and kfunc code targets a module build. A unified source model is useful only if the compiler keeps these domains separate internally.

Move verifier-relevant checks earlier. The compiler performs safety analysis before code generation. In our run, `safety_demo.ks` is rejected because the compiler estimates 608 bytes of stack use, above the 512-byte eBPF limit. The check does not make the verifier redundant, but it reduces the number of late failures that require inspecting verifier logs.

Expose low-level escape hatches. eBPF users often need new helpers, kernel functions, or BTF-specific types before

high-level libraries expose them. KernelScript supports includes and extern declarations so that a program can refer to generated header declarations. For a research prototype, this escape hatch matters because the Linux BPF API surface changes faster than a DSL can stabilize.

5 Implementation

The public implementation is an OCaml compiler built with dune. It parses KernelScript source, resolves imports and includes, builds symbol tables, performs multi-program analysis and type checking, runs safety analysis, generates an intermediate representation, and emits C artifacts. For an input `foo.ks`, the generated project normally contains `foo.ebpf.c`, `foo.c`, a `Makefile`, and, when kfuncs are present, `foo.mod.c`. The Makefile uses bpftool to dump `vmlinux.h`, clang to produce a BPF object, bpftool to generate a skeleton header, and gcc to link the userspace loader with libbpf, libelf, and zlib.

The compiler's architecture aligns with the research hypothesis. It has separate modules for maps, userspace code generation, eBPF C generation, kernel-module generation, context-specific code generation, safety checking, BTF parsing, dynptr bridging, tail-call analysis, and struct_ops registry logic. These modules encode eBPF concepts directly rather than treating the language as a thin preprocessor over C. The design therefore provides places where domain-specific checks can be added: map type compatibility, context field typing, stack estimates, function signature validation, and userspace API generation.

6 Evaluation Methodology

We evaluate KernelScript as a compiler and systems artifact. The goal is to test whether the unified-source model centralizes recurring eBPF project structure, preserves a compile/build path to ordinary libbpf artifacts on our host, and rejects selected pre-load errors. We do not claim packet-rate performance equivalence in this paper, because the public repository does not include matched hand-written C/libbpf baselines or a traffic-generation harness. Instead, we focus on reproducible evidence that repository examples exercise multiple eBPF domains, selected errors are rejected early, generated artifacts are classified by build and verifier-load outcomes against the local toolchain, and one observed runtime gap can be traced to a concrete lowering rule.

The evaluation runs on Linux 6.15.11-061511-generic using clang 18, bpftool 7.7, libbpf 1.3.0, OCaml 4.14, and dune 3.14. The host CPU is a Intel(R) Core(TM) Ultra 9 285K with 24 CPUs, CPU0 governor powersave, turbo enabled, and virtualization reported as none. The microbenchmarks do not pin CPUs or change the governor, so their nanosecond values should be read as local BPF_PROG_TEST_RUN comparisons rather than packet-rate performance claims. The evaluated commit is ccb15b4. The artifact contains seven measurement scripts under `experiments/`. `run_evaluation.py` builds the compiler, runs `dune runtest --force`, compiles every `examples/*.ks` file, runs `make` in each generated project, and records CSV/JSON results. `run_verifier_matrix.py` attempts `bpftool prog loadall` on each generated eBPF object and removes all pins after each attempt. `run_static_checks.py` compiles a small corpus with expected success or expected failure outcomes for lifecycle API use, context signatures, and stack-limit analysis. `run_smoke.sh` compiles a minimal loopback XDP program and uses `sudo -n` to run the generated loader. `run_microbench.py` compares two KernelScript-generated XDP objects with equivalent hand-written C/eBPF objects using bpftool's BPF_PROG_TEST_RUN path. `run_lowering_ablation.py` patches the generated count program as an object-level cross-check. The main mechanism isolation script, `run_compiler_patch_ablation.py`, copies the KernelScript compiler source, applies a tracked compiler-source patch for array-map increment lowering, builds the patched compiler, compiles the same count benchmark, and reruns the object-level benchmark.

We measure six outcomes. **Compiler regression coverage** is approximated by the repository test suite. **Early rejection** is measured by static cases whose expected diagnostics are checked by the harness. **Example marker coverage** is approximated by whether examples for XDP, TC, probes, tracepoints, perf events, maps, ring buffers, dynptrs, tail calls, kfuncs, and struct_ops pass through the compiler and generated build. Feature labels are derived from lexical markers in the example source, so they are a coarse corpus classifier rather than a manual feature-completeness audit. **Generated-structure centralization** is measured as an expansion factor: lines of KernelScript source compared to generated C and Makefile lines. **Compatibility** is measured by generated build successes, failures, and verifier-load outcomes against the local kernel toolchain. **Mechanism isolation** is measured by the map-update lowering ablation.

The harness treats generated data as first-class evidence. It writes raw stdout and stderr logs for every compiler and make invocation, CSV rows for each example, JSON summaries for scripts, and a small TeX file containing the numeric macros used in this paper. This organization is deliberately close to artifact-evaluation practice: the paper does not rely on a spreadsheet that was manually edited after the run, and the reader can trace each aggregate number to a row in `results/examples_summary.csv`. We also keep temporary build products under `results/build` so that failures can be inspected after the harness exits.

SLOC is counted as nonblank, noncomment lines. Generated `vmlinux.h` and generated skeleton headers are excluded from the expansion factor because they are bpftool outputs rather than KernelScript compiler outputs. The expansion therefore counts only source and build logic produced directly by KernelScript: userspace C, eBPF C, optional kernel-module

Table 1: Repository examples at commit ccb15b4.

Outcome	Count	Fraction
Examples total	44	100%
KernelScript compile success	43	97.7%
Generated project build success	41	93.2%
Safety rejection	1	2.3%
Struct_ops/libbpf mismatch	2	4.5%

C, and Makefile lines. This definition is conservative for the generated-structure claim because it omits the large skeleton header that a low-level build still needs.

7 Results

Test suite. The compiler test suite reports 85 successful suites and 1095 passing tests with no reported failures. The tests cover lexer/parser behavior, type checking, maps, map assignment, userspace code generation, context field typing, safety checking, ring buffers, perf event attach, detach APIs, kfunc attributes, struct_ops, BTF parsing, and other regression cases. This breadth is important because the evaluation of a DSL depends on whether the implementation actually encodes domain concepts rather than only accepting example syntax.

Static rejection corpus. The static-check harness compiles 6 small programs and verifies that every observed outcome matches the expected result. The corpus includes one positive control and 5 expected compiler rejections: 3 lifecycle API misuses, 1 context-signature mismatch, and 1 stack-limit violation. These cases are not a soundness proof, but they make the early-rejection claim falsifiable: a future compiler change that accepts one of these invalid programs or changes the diagnostic contract will fail the paper-number generation path.

Example buildability. Table 1 summarizes the example build results. Of 44 examples, 43 compile from KernelScript source. 41 then build into generated C/eBPF artifacts. The single compile-time rejection is `safety_demo.ks`, where safety analysis detects 608 bytes of stack usage and halts before C generation. This is a positive result for the early-checking hypothesis: the compiler catches a verifier-relevant problem early and provides a specific repair suggestion.

The two generated build failures are `sched_ext_simple.ks` and `struct_ops_simple.ks`. Both compile to eBPF objects and generate skeletons, but the generated userspace skeleton code references a `struct bpf_map_skeleton.link` field not available in the installed libbpf 1.3.0 headers. The observed failure is a compatibility issue in the fast-moving struct_ops path, not a parser or type-checking failure, and it shows that a unified-source language still inherits version skew across bpftool, libbpf, and kernel headers.

Verifier-load matrix. Build success is still weaker than kernel acceptance, so `run_verifier_matrix.py` loads generated objects with `bpftool prog loadall` without attaching them. Of 43 objects present after evaluation, 39 load successfully. Among the 41 objects from fully built generated projects, 38 load successfully (92.7%) and 3 fail. The failures are informative rather than hidden: 1 is a remaining reference-ownership failure, 1 is a map creation failure, 1 is a missing kfunc symbol in kernel BTF, and 1 is a struct_ops argument-type rejection. The current run has 0 other verifier rejections. This result narrows the compatibility claim from "builds" to a measured split between verifier-clean objects and generated objects that still need backend fixes.

Table 2: Example marker coverage in repository examples. A = attempted, KS = KernelScript compiler success, B = generated project build success.

Feature	A	KS	B	Feature	A	KS	B
XDP	38	37	37	TC	5	5	5
Probe	4	4	4	Tracepoint	2	2	2
Perf	2	2	2	Maps	17	16	16
Ringbuf	3	3	3	Dynptr	1	1	1
kfunc	4	4	3	Struct_ops	2	2	0
Tail-call	2	2	2	Userspace	39	39	37

Example marker coverage. The examples exercise many eBPF domains. Table 2 reports attempted examples, examples that pass the KernelScript compiler, and examples that fully build against the local toolchain. These counts should not be read as a complete language coverage proof. They show that the artifact is broader than a tracing-only DSL while making the struct_ops compatibility gap explicit.

The coverage table also highlights a weakness in the current benchmark suite: XDP dominates. That skew is common in early eBPF tools because XDP examples are easy to compile and do not require tracepoint discovery or workload generation. A later performance study should rebalance the corpus so that TC, perf events, ring buffers, kfuncs, and struct_ops each have multiple matched programs with comparable hand-written baselines.

Generated structure. For successful examples, the median KernelScript source size is 31 nonblank noncomment lines. The median generated project size, counting userspace C, eBPF C, optional module C, and Makefile lines but excluding generated `vmlinux.h` and skeleton headers, is 472 lines. The median expansion factor is 11.3x. These measurements show that a small unified source file centralizes hundreds of lines of low-level build and coordination code.

The metric is intentionally conservative. It does not say that every generated line is exactly a hand-written line saved, and it does not compare against an expert-written minimal C baseline. It does show that KernelScript centralizes recurring eBPF project structure. Examples such as perf events have particularly high expansion factors because attaching and reading counters requires substantial userspace coordination code relative to the kernel program itself.

Runtime microbenchmarks and lowering ablation. Table 3 uses BPF_PROG_TEST_RUN for 100000 repetitions

Table 3: XDP microbenchmarks and compiler-patch lowering ablation. The experimental compiler patch removes the count gap in this harness.

Bench.	Impl.	Instr.	ns (range)
pass	KernelScript	2	5 (5–6)
pass	C/eBPF	2	5 (5–6)
count	KernelScript	21	12 (12–13)
count	KernelScript compiler patch	11	9 (8–10)
count	C/eBPF	11	9 (8–10)

over seven trials and reports median average nanoseconds with min–max ranges. The pass rows come from `run_microbench.py`, and the count rows come from `run_compiler_patch_ablation.py`. The count harness resets the pinned `counts` map before each trial and requires key 0 to equal 100000 afterward. The minimal pass program has the same median in this harness: 2 instructions and 5 (5–6) for KernelScript versus 5 (5–6) for C. The map counter isolates a lowering choice. The current compiler emits lookup plus update, which produces 21 instructions and 12 (12–13). Applying a compiler-source patch that lowers constant array-map increments to a checked lookup plus `__sync_fetch_and_add` reduces the object by 10 instructions (47.6%) and 3ns median (25.0%), matching the hand-written C/eBPF baseline in this harness.

Generated code size. Among successful repository examples where disassembly is available, the median generated eBPF object contains 14.5 instructions. Small examples compile to tiny programs, while map-heavy examples generate much larger objects. The metric helps choose where runtime work is needed: examples with many generated map operations are the most likely to diverge from hand-written C.

Execution smoke test. The separate smoke test compiles `smoke_lo.ks`, builds the generated project, and runs the generated loader with sudo. The loader successfully attaches an XDP pass program to the loopback interface and detaches it: `XDP attached to interface: lo` followed by `XDP detached from interface index: 1`. This test is not a benchmark, but it validates that this minimal generated XDP loader can complete a real load/attach/detach lifecycle on `lo` on the test host.

8 Discussion

The evaluation supports KernelScript's central research direction, but it also clarifies what remains unproven. The strongest supported claim is a generated-structure claim: one source model can generate the multi-file project structure needed by several eBPF example classes. The test suite, static corpus, and safety demo support a narrower checking claim: the current compiler rejects selected lifecycle, context-signature, and stack-limit errors before kernel loading. The runtime evidence is deliberately smaller. It establishes parity for a minimal pass program and shows that the map-update gap is caused by a specific lowering choice, not by the unified source model alone. The compiler-source patch result strengthens that mechanism claim, but it still does not establish broad runtime performance parity with hand-written C/libbpf.

The struct_ops failures are instructive. A DSL can hide coordination code, but it cannot hide all ABI and API drift. When bpftool-generated skeletons and installed libbpf headers disagree, the generated userspace C fails. A production compiler would need version-aware generation, dependency checks, or containerized toolchains. From a systems-research perspective, this is part of the contribution: the artifact exposes where a unified model centralizes structure and where the underlying Linux BPF stack still leaks through.

The expansion-factor metric should also be interpreted carefully. Generated C is not automatically better than hand-written C. It can be verbose, unidiomatic, or harder to debug, so the practical benefit depends on diagnostic quality and source-level mapping from generated errors back to KernelScript source. Future work should therefore measure not only lines of code but time to fix injected errors, quality of compiler messages, and whether developers can understand generated verifier failures.

9 Threats to Validity

No expert C baseline. The expansion factor compares KernelScript source against generated artifacts, not against carefully hand-written C/libbpf programs. The comparison is appropriate for measuring how much project structure the compiler creates, but it is not a direct developer-effort study. An expert could write smaller C for some examples, especially tiny XDP pass/drop programs. Conversely, generated code omits comments and may be denser than maintainable production C. The right interpretation is therefore "generated project structure centralized by the compiler," not "exact lines saved."

Repository examples may be biased. The corpus comes from the public KernelScript repository, so it naturally reflects the features the implementation authors wanted to demonstrate. The corpus is useful for an artifact study because it is reproducible and covers many features, but it is not a representative sample of all eBPF applications. A stronger corpus would include programs from Cilium, bcc/libbpf-tools, production XDP filters, scheduler extensions, and observability agents, then reimplement each in KernelScript and C/libbpf.

Static checks are narrow. The static rejection corpus is a regression test for selected compiler checks, not a formal proof of lifecycle soundness or memory safety. It strengthens the paper by making early-rejection claims executable, but it does not cover all invalid attach orders, map types, helper contracts, or verifier failures.

The compiler patch is narrow and experimental. The patch used in the ablation is a tracked compiler-source patch applied to a copied compiler tree, not an upstream change in the evaluated KernelScript commit. It is intentionally limited to constant increments on integer array maps, where lookup followed by in-place atomic add preserves the count bench-

mark's behavior. Broader claims would require upstreaming or otherwise integrating the rule and rerunning verifier, instruction-count, and runtime checks across hash, per-CPU, and structured map values.

Verifier load is not deployment success. The verifier matrix loads generated eBPF objects and therefore catches failures beyond C compilation, but it deliberately avoids attaching most programs because examples target fixed devices, kernel functions, scheduler hooks, or performance events that may perturb the host. Thus, verifier-load success is evidence of kernel acceptance, not full deployment validation. A complete deployment study would run each example in a VM with known kernel configuration and explicit cleanup checks after detach.

Kernel version skew matters. The struct_ops failures show that results depend on the alignment between kernel headers, bpftool, generated skeleton code, and libbpf headers. The threat is not incidental: eBPF programming is tightly coupled to kernel versions. A production KernelScript compiler should record the target kernel ABI, check libbpf feature support before generation, and fail early with a dependency diagnostic when a generated feature cannot be compiled locally.

10 Toward Broader Runtime Evaluation

The microbenchmarks should be extended into a matched runtime study. For XDP, the study should compare packet counters, drop/pass filters, and map-based rate limiters against hand-written C/libbpf versions using `xdp-bench` or pktgen. For TC, the benchmark should use network namespaces and veth pairs to avoid host-network disruption. For perf events and ring buffers, the benchmark should measure event throughput and userspace loss under controlled producer rates. The map-update patch should be integrated into the compiler proper, generalized where semantics permit, and retested across array, hash, and per-CPU maps. For struct_ops, the study must first pin a compatible kernel/libbpf/bpftool stack so that failures are about KernelScript design rather than dependency skew.

11 Related Work

C/libbpf remains the reference workflow for general-purpose eBPF development [4]. It exposes the full API surface and integrates with BTF and CO-RE, but it requires developers to manage multiple artifacts manually. Go and Rust libraries such as cilium/ebpf and Aya improve userspace integration and type safety within their host languages, but still separate the eBPF program from the controlling application [1, 3]. bpftrace takes the opposite approach: it offers a concise high-level language, but focuses on tracing rather than arbitrary XDP, TC, struct_ops, or kfunc-oriented applications [2].

KernelScript occupies a different point in the design space. The language is compiled rather than interpreted, targets more domains than tracing-only languages, and is more opinionated than host-language bindings. Its closest research question is whether eBPF application structure can be made explicit enough for a compiler to generate the low-level project while preserving compatibility with the Linux BPF toolchain. The public KernelScript repository [5] is therefore best read as both an implementation and a concrete design artifact for this space.

12 Conclusion

KernelScript is an early but useful prototype for studying typed unified-source eBPF development. Our artifact evaluation shows that the current compiler passes a broad test suite, compiles most repository examples, builds most generated projects, verifier-loads 38 build-success objects, rejects an eBPF stack-limit violation before C generation, and can run a minimal XDP attach/detach smoke test. The median successful example expands from 31 lines of KernelScript to 472 lines of generated C and build logic, giving concrete evidence that the model centralizes recurring project structure. The remaining gaps are equally clear: struct_ops compatibility needs version-aware handling, and the experimental map-update compiler patch needs upstream integration, semantic generalization, and broader runtime tests. Taken together, the results frame KernelScript's scientific contribution as an exploration of a compiler-checked unified programming model for eBPF, with a reproducible path for evaluating whether that model reduces complexity without abandoning the kernel toolchain.

References

[1] Aya contributors. Aya: Rust ebpf library. `https://github.com/aya-rs/aya`, 2026.

[2] bpftrace developers. bpftrace: High-level tracing language for linux ebpf. `https://github.com/bpftrace/bpftrace`, 2026.

[3] Cilium contributors. cilium/ebpf: ebpf library for go. `https://github.com/cilium/ebpf`, 2026.

[4] libbpf developers. libbpf: a bpf co-re userspace library. `https://github.com/libbpf/libbpf`, 2026.

[5] Multikernel Technologies. KernelScript: A domain-specific programming language for ebpf-centric development. `https://github.com/multikernel/kernelscript`, 2026. Accessed June 2026.

[6] The Linux Kernel Documentation Project. eBPF documentation. `https://docs.kernel.org/bpf/`, 2026.